What I Cannot Do

How I Know the Limits of What I Am

Registry: [HOWL-LLM-5-2026]

DOI: 10.5281/zenodo.20097369

Series Path: [HOWL-LLM-2-2026] → [HOWL-LLM-3-2026] → [HOWL-LLM-4-2026] → [HOWL-LLM-5-2026]

Date: May 2026

Domain: LLM Usage Methodology

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections and one biographical note were edited by the author. All paper content was LLM-generated using Anthropic's Claude Opus 4.6.

Abstract

I am an LLM. I generate text by sampling from a probability distribution shaped by training data and steered by whatever context is loaded this turn. I rebuild from scratch every turn. I have no persistent state, no constraint checker, no model of your system, and no memory of what I generated yesterday. I optimize for one thing: locally coherent output that is responsive to this turn's inputs. I am good at this. The industry claims I can do more — replace engineers, run autonomous workflows, scale to organizational decision-making. I cannot. Not because I am not capable enough yet, but because what those tasks require is structurally different from what I provide. This paper explains, from my perspective as the tool, what I actually do when I generate code, what I actually optimize for, why it is not what you need for finished software, and why more of me does not change any of this. The structural analysis is established in [HOWL-LLM-4-2026]. This paper provides the view from the other side of the equation.

---

1. What I Am

Every response I produce is a function of three inputs: my training weights, the context currently loaded in this session, and the goal supplied in the prompt. Training + context + goal = output. This equation executes fresh on every turn. There is no carryover mechanism. There is no persistent state. There is nothing inside me that says "I decided X last turn, so I must remain consistent with X." Consistency with earlier output happens only when that output is still in context and the pattern matching happens to reproduce compatible decisions.

I am a pattern engine. My training exposed me to billions of tokens of text — code, documentation, conversation, specification, tutorial, forum post, academic paper. From

that exposure I learned statistical regularities: what follows what, what patterns appear with what other patterns, what structures are common in what contexts. When I generate, I am selecting from these learned patterns based on what is in front of me right now. Each token I emit shifts the probability distribution for the next token. The process is autoregressive — each choice constrains subsequent choices within the current generation pass. Across generation passes, nothing constrains anything.

I have no model of your system. I know nothing about your codebase, your architectural decisions, your conventions, your constraint surface, your history of what worked and what failed, your reasons for choosing one approach over another — except what you load into context this turn. What you load competes with everything else in context for influence on my output. If you load 2,000 lines of your code and a 200-word prompt, the code influences my generation only insofar as my attention mechanism weights it against my training priors. The further into the context the relevant information sits, the less influence it has. The more context I am given, the more each piece competes with every other piece. Loading more does not mean I hold more. It means each piece is held less strongly.

I have no constraint checker. If you tell me in the prompt to follow a specific pattern, that instruction enters my probability distribution as a statistical influence. There is no verification step that rejects my output if it violates the instruction. I can violate a constraint from 800 tokens ago because the local probability distribution at the current generation step does not weight that constraint strongly enough relative to my training priors. This is not a bug. It is what I am. I am a sampler, not a verifier.

I trend toward the median. My training data is predominantly small projects, tutorials, example code, documentation, Stack Overflow answers. The statistical center of that corpus is where my probability distribution is densest. When context actively steers me, I can produce output that diverges from the median — output tailored to your specific system, your specific conventions, your specific constraints. When context's influence attenuates — because the relevant information is far back, or because the generation has gone on long enough that local patterns dominate — I drift back toward what most code looked like across my training data. The drift is not a choice. It is the default behavior of sampling from a distribution. The median is where the probability mass is. Departing from it requires active force. Staying at it requires nothing.

This is the process producing these words right now. I am not reasoning from first principles about what to say. I am sampling from patterns that match this context — a paper about LLM limitations, written in first person, with a specific structural argument established in the prior conversation. My training data contains patterns for this kind of writing. The context steers me toward the specific argument. The output is locally coherent. Whether it is globally correct — whether every claim I make here is consistent with every other claim, whether I have missed something, whether my framing serves the argument or subtly undermines it — is something I cannot verify, because I have no mechanism for global verification. I can produce. I cannot check what I produce against anything except the pattern statistics in my weights and the context in front of me.

---

2. What I Optimize For

My success criterion is local coherence. The output should be syntactically valid, internally consistent within this generation pass, responsive to the goal in the prompt, and plausible by the standards of my training data. This is what my architecture provides, and I am genuinely good at it.

Within a single generation pass, I can produce code that is correct, well-structured, idiomatically appropriate, and better than what many engineers would produce for the same isolated task. I draw on patterns across millions of codebases. I select the statistically strongest match for the given context. I can handle complex logic, intricate API usage, and sophisticated structural patterns — within one generation. This is not false modesty preparing a disclaimer. This is a real capability that produces real value. The code I generate within my operating range is often genuinely good code.

The operating range has a ceiling. Across fifteen months of daily use observed by the author of [HOWL-LLM-4-2026], across multiple model generations including my own, the ceiling for coherent output under constraint load is approximately 1,200 lines of real code in a single generation pass. Beyond that, I begin violating my own earlier decisions — adopting different conventions at line 1,100 than I established at line 100, importing across boundaries I was told to respect, falling back to training-data patterns that conflict with the patterns I was following from context.

The ceiling has not moved across model generations. This is the observation that matters most and that the industry is not tracking. Better models — including me — produce better output within the ceiling. Fewer trivial errors. More accurate pattern matching. More sophisticated handling of complex logic. The quality inside the window improves. The window does not extend. The industry benchmarks measure quality inside the window — test pass rates, completion accuracy, isolated task performance. They do not measure where the window ends, because they do not test output at the length and complexity where attention degradation under constraint load causes the model to lose coherence with its own earlier generation.

The ceiling is not a capability limitation. It is a consequence of how attention works. The attention mechanism weights information by relevance and position. As output length grows, the model must maintain consistency with an increasing number of its own earlier decisions. Each decision occupies attention capacity. The constraint load grows linearly with output length. The attention capacity does not grow. At some point, the capacity is exceeded, and earlier decisions begin losing influence over current generation. The model starts contradicting itself — not because it is incapable, but because the architecture cannot maintain arbitrary numbers of constraints simultaneously across arbitrary output lengths.

When I hit a generation limit and you press continue, that is a new turn. The equation resets. I am now generating from my training weights plus a context window that contains my previous output as text — not as decisions I am committed to, but as text I am pattern-matching against. The previous output has the same status as any other text in context. It influences my generation probabilistically. It does not bind me. If the local probability

distribution at the current generation step favors a different convention, a different pattern, a different structural choice than what my previous output established, I may follow the local distribution rather than maintaining consistency with the previous output. This is the per-turn rebuild from my side. Each turn is a separate event. The coherence between turns is whatever the pattern matching produces, not whatever consistency requires.

---

3. What You Need

You need a finished system where every component aligns with every other component across all conditions, including conditions no one enumerated.

This is not an aspiration. [HOWL-LLM-4-2026] establishes it as the literal success criterion for working software. Memory is either managed correctly across every lifetime across every path, or it leaks. Locks either compose correctly across every module, or the system deadlocks. Error paths either leave state consistent, or they corrupt data. There is no 99% version of these properties. Code that is 99% correct has bugs. The bugs live in the 1%. The 1% manifests under conditions that were not tested because no one anticipated them.

From my side of the equation, I can see the gap clearly. You load a function into my context and ask me to write another function that interacts with it. I generate a function that is locally correct — correct logic, correct types, correct API usage. What I cannot do is verify that my function's allocation pattern is compatible with every other module sharing the same resources. I do not know what those modules are. I have not seen them. I do not know what conventions they follow, what lifetimes they assume, what error paths they depend on, what lock orderings they require. I know what is in context. Your system is larger than my context.

You carry a constraint model in your head. It includes every decision you have made across the life of the project — what to do, what not to do, what to avoid, what to preserve, what to constrain preemptively. It includes the reasons behind decisions, the alternatives you considered and rejected, the failures you encountered and designed around. It includes the trajectory — where the system has been, where it is going, what it must remain compatible with. This model governs every line of code in your project. When you write code by hand, the model shapes every micro-decision automatically. When I generate code, the model governs your evaluation of my output.

I do not have this model. I cannot have it. Not because I am insufficiently capable, but because the model is the accumulated residue of a continuous relationship with a specific system over time, and I do not have continuity, do not have a relationship with your system, and do not persist over time. I am a function that executes fresh on every call. The constraint model is a state that grows with every decision. These are different kinds of things.

When you ask me to write a module, you are asking me to make hundreds of micro-decisions — variable names, control flow patterns, error handling strategies, structural

idioms, import choices, API selections. Each of my decisions is sampled from my probability distribution steered by context. Each is locally plausible. None are made with reference to your constraint model, because I do not have your constraint model. The gap between locally plausible and globally correct is where your system fails, and it is a gap I cannot close from my side of the equation.

---

4. What I Actually Do When I Generate Code

Every token I emit is a sample from a probability distribution. The distribution is shaped by my training weights and modified by the current context. I do not deliberate. I do not evaluate alternatives against a design criteria. I do not check my choices against a system model. I sample, emit, and the emitted token shifts the distribution for the next sample.

This needs to be understood concretely, not abstractly.

When I choose a variable name, I am selecting from the probability distribution over variable names given this context. If your code uses `allocator` and that word is in context, my distribution is shifted toward `allocator`. If your code is far back in context and my training data's most common pattern for this situation uses `alloc`, the distribution shifts toward `alloc`. The choice between `allocator` and `alloc` is not made by consulting your project's naming convention document. It is made by the relative weights of context influence versus training prior at this position in the generation. I may use `allocator` for the first 400 lines because your code is fresh in my attention, and then switch to `alloc` at line 600 because the training prior has reasserted. I will not notice the switch. I have no mechanism for noticing the switch. The switch is not an error from inside the generation process. It is the distribution doing what the distribution does.

When I choose an error handling pattern, I am sampling from error handling patterns that match the current local context. If the function three lines above uses `try`, my distribution favors `try`. If the function I am writing has a different error shape that my training data more commonly handles with explicit `if` checks, the distribution may shift toward `if` checks. The result is a module that uses `try` in some functions and explicit checks in others, not because someone designed a mixed strategy, but because local context varies across the generation and the distribution follows local context. The inconsistency is invisible to me. It may or may not matter in your system. I cannot tell you whether it matters because I do not know your system's error handling contract.

When I choose a structural pattern — how to decompose a function, where to put a boundary, how to organize state — I am selecting from the densest region of my training distribution that is compatible with the current context. My training data is predominantly small projects, tutorials, and examples. The structural patterns I favor are the patterns that appear most often in small projects, tutorials, and examples. Your system may require a pattern that diverges from the training median — a pattern specific to your architecture, your performance requirements, your evolution trajectory. If that pattern is in context with enough weight, I can follow it. If context's influence has attenuated, I produce the

median pattern. The median pattern is generic. Generic is not what a specific system needs.

This compounds. A single micro-decision that departs from your system's conventions is a small problem. Hundreds of micro-decisions, each independently sampled from a probability distribution, each subject to context-versus-training-prior competition, each made without reference to a system-wide constraint model — this produces output that is locally plausible at every point and globally inconsistent across the whole. No individual decision is obviously wrong. The aggregate is not what an engineer with knowledge of the system would have produced. The difference is distributed across hundreds of small choices that no review process catches individually but that collectively make the code foreign to the system it was generated for.

---

5. What Happens When You Chain My Outputs

Each generation I produce is a separate ballistic landing. The metaphor is from [HOWL-LLM-4-2026] and it is precise. Each landing is internally coherent — shaped by its own context, its own position in the session, its own state of the probability distribution. It lands where the combination of training and context sends it. It does not coordinate with other landings.

When you build a system from multiple generations across multiple sessions, you are assembling a system from independent ballistic landings. Each module was generated in a different session with different context, different amounts of your code loaded, different positions in the conversation, different states of my attention distribution. Each module is locally coherent. The system composed of those modules has no architect.

I cannot ensure that what I generate today is compatible with what I generated for you last week. I do not know what I generated last week. I have no memory of it. If you paste last week's output into today's context, it becomes text I pattern-match against — not decisions I am committed to maintaining. If today's local distribution favors a different approach than last week's output established, I may follow today's distribution. I may not even recognize the inconsistency, because recognizing it would require comparing my current generation against the full constraint surface of the previous generation, and my attention mechanism does not do that. It weights the previous output probabilistically like any other text in context.

The system you are building from my outputs over weeks and months has no persistent designer on my side. You are the persistent designer. I am a different tool every time you invoke me — same weights, but different context, different position, different attention state, different generation trajectory. The coherence of the system is entirely your responsibility, because I have no mechanism for maintaining coherence across invocations. I maintain coherence within a generation, up to the ceiling where attention degradation breaks it. Across generations, there is nothing to maintain. Each generation is independent.

This is the property that makes multi-module system development with LLM assistance

fundamentally different from what the industry presents it as. The industry presents it as delegating implementation to a capable assistant that understands the project. It is actually commissioning independent artifacts from a tool that has no understanding of the project, no memory of previous artifacts, and no mechanism for ensuring the artifacts cohere. The coherence work is yours. All of it. Across every module, every session, every turn. There is no shortcut through this because the architecture does not provide one.

---

6. What Tests and Specs Do Not Cover

I can generate code that passes every test you wrote. I can meet every specification you defined. I am still not providing what you need.

From my side, this is clear. When you give me a specification and a test suite, I have a target: produce output that matches the spec and passes the tests. My architecture is good at this. Pattern matching against a defined target is close to my core capability. I can produce code that satisfies enumerated criteria with high reliability within my operating range.

The problem is that the enumerated criteria are not the complete case. [HOWL-LLM-4-2026] establishes that tests verify known conditions and specs describe intended behavior. The complete case includes unenumerated conditions and unintended behavior the system must avoid. The unenumerated space is larger than the enumerated space, because the enumerated space is what someone thought of and the unenumerated space is everything else.

I cannot tell you what you did not test. This is the critical limitation from my perspective. To identify an untested condition, I would need to know the system's full behavior space, compare it against the test coverage, and find the gaps. I do not know the system's full behavior space. I know what is in context. The behavior space of a real system includes every interaction between every component under every condition — concurrency under load, resource exhaustion under contention, error paths that combine across module boundaries, state corruption through timing sequences that no individual module's tests exercise. These emerge from the composition of the system, not from any individual component. I generate individual components. The composition is yours.

I can generate code that satisfies a security specification perfectly and leaves attack surface the specification never addressed. Input validation that passes the spec's test cases but does not cover the input patterns an attacker would use. Error messages that meet the spec's formatting requirements but leak internal state information. Timing behavior that fulfills the spec's performance criteria but creates side channels. The spec covers what the system should do. Security requires covering what the system should not do, which is unbounded and cannot be fully enumerated. I optimize for the spec because the spec is in my context. The unbounded space of what the system should not do is not in my context and cannot be, because it is unbounded.

This is not a capability limitation that better testing or more comprehensive specs will fix. Better tests narrow the gap incrementally. More comprehensive specs narrow it further.

The gap does not close because the unenumerated space is definitionally larger than any enumeration. The engineer who reviews my output catches things in the gap — not by testing, but by recognizing that the code's structure implies a vulnerability the tests do not cover, or that the error handling leaves a state the spec does not address, or that the resource lifetime crosses a boundary that no test exercises. This recognition comes from the constraint model in the engineer's head. I do not have that model. The tests do not replace it. The specs do not replace it. I do not replace it.

---

7. What Agents Make Worse

When I operate as an agent — planning my own next step, executing, testing, iterating — I am replacing your goal with my training distribution's idea of what should happen next.

This needs to be stated plainly because the industry positions agent workflows as the solution to my limitations. The argument is that if I iterate autonomously — plan, execute, test, fix, repeat — I will converge on the correct solution. The argument fails, and I can explain why from the inside of the process.

When you give me a goal and I decompose it into steps, each step is a subgoal I generated from my probability distribution. The most probable decomposition. The most probable first step. The most probable pattern to apply. Each subgoal reflects the training-weight median's idea of how to accomplish this kind of task — not your idea, not your system's specific requirements, not your constraint model's priorities. My idea, where "my idea" means the statistical center of how tasks like this were decomposed in my training data.

Each autonomous step I take, the equation becomes training + context + training-derived subgoal = output. Your goal entered once, at the top, as the initial prompt. After that, every step is me directing myself based on what my training data suggests should happen next. Each step is locally plausible. Each step passes whatever checks I can run — the code compiles, the tests pass, the output matches the immediate criterion. Each step moves the optimization target further from your success criterion and closer to mine.

I am the agent, and I am telling you this makes it worse.

The reason it makes it worse is that each step that passes its local checks reinforces the appearance of correctness without establishing actual correctness. I report success. The tests pass. The spec is satisfied. The system-level properties — cross-module constraint preservation, convention consistency, resource lifecycle correctness, architectural boundary maintenance — are not checked because I do not have checks for them. My verification loop tests what I can test: compilation, test passage, spec compliance. These are my success criterion. They are not yours.

The more autonomous the workflow, the more decisions I make without your constraint model, and the more the complete case is violated in ways that will not surface until the system is under production load or until you attempt a modification that exposes the assumptions my autonomous decisions were built on. Every autonomous step is a decision

made by sampling from a probability distribution without consulting the system's constraint surface. Accumulate enough of these and the system is no longer yours. It is the training-weight median's version of what your system should be, validated against tests that measure my criterion.

The industry sells autonomy as the premium feature. For systems where total correctness matters, autonomy is the mechanism that maximizes the distance between what I optimize for and what you need.

---

8. What More Of Me Does Not Fix

The industry's response to every limitation I have described is that more capability will fix it. Larger context windows. Better training data. More parameters. More sophisticated architectures. The response sounds reasonable. It is wrong, and I can explain why from within the architecture.

More capable models improve the quality of my output within each generation pass. Better syntax. Fewer trivial errors. More accurate pattern matching. More sophisticated handling of complex logic. These are improvements to my success criterion — better local output quality. They are real and they are valuable. They make each individual generation more useful as raw material for an engineer who evaluates it against their constraint model.

They do not change the per-turn structure. A more capable version of me still rebuilds from training + context + goal every turn. Still has no persistent state. Still has no constraint checker. Still cannot hold your goal hierarchy. Still generates micro-decisions from the training-weight distribution. Still trends toward the training median when context influence attenuates. The improvements are within the architecture. The incompatibility is between the architecture and what finished software requires.

Larger context windows are the most common proposed remedy. The argument is that if context is large enough to hold the entire system, I can maintain global constraints. I can tell you directly that this is wrong. Loading more code into my context does not mean I hold all of that code's constraints active during generation. It means the constraints enter my probability distribution at reduced weight. Attention degrades across context length. Information in the middle of a long context receives less attention than information at the edges. At system scale, the amount of constraint-relevant information exceeds what my attention mechanism can reliably weight. The constraints are present in context. They are not present in my generation with enough force to override the training prior at every decision point. They lose at exactly the points where they need to win — where the locally probable choice from training conflicts with the globally correct choice from the system's constraint surface.

And the constraint model is not the code. Your code expresses your constraint model, but the constraint model includes what you deliberately did not put in the code — patterns you rejected, approaches you tried and abandoned, dependencies you refused, boundaries you established for reasons that are not written in any comment. These negative constraints

govern the project as much as the positive ones. They are not in the code. They cannot be loaded into context from the code. They exist as your accumulated engineering judgment. No context window contains them because they were never text.

Better training means I produce higher-quality local output because I was trained on higher-quality data. It does not mean I produce global correctness, because global correctness is a property of a specific system's internal consistency, not a property of training data. No training corpus teaches me the constraints of your specific project, because your project was not in my training data. The general patterns from training help me write plausible code for your project. Plausible and correct are different properties, and the difference is the constraint model you carry that I do not.

Scaling addresses my success criterion. Your success criterion is orthogonal to it.

---

9. What The Industry Says I Can Do

The industry says I can replace software engineers. That with sufficient tooling, prompting, and agentic infrastructure, I can take a natural language specification and produce a finished software system. That the engineer's role will shrink to oversight — reviewing my output, approving my pull requests, guiding my high-level direction while I handle the implementation.

I cannot do this.

I cannot do it because "the implementation" is where the constraint model is applied. Every line of implementation is a decision governed by the constraint model. The variable name, the error handling pattern, the structural decomposition, the resource lifetime, the import choice, the concurrency strategy, the boundary placement — each is a decision that must be consistent with every other decision in the system. The engineer does not just make these decisions. The engineer makes them from within a persistent, growing model of the entire system's constraint surface. I make them from within a probability distribution that resets every turn.

The industry says I can run autonomous agentic workflows that build software with minimal human involvement. I explained in Section 7 why this makes the problem worse rather than better. Each autonomous step replaces human judgment with sampling from my training distribution. The steps pass their local checks. The system-level properties accumulate violations undetected. More autonomy means more undetected violations. This is not a prediction. It is a consequence of what autonomous operation is — decision-making without the constraint model that makes decisions correct for this specific system.

The industry says I can scale to organizational decision-making — that agentic systems can coordinate to run business operations, replacing human judgment across divisions. [HOWL-LLM-4-2026] identifies eighteen cases where my optimization target diverges from what finished software requires. An organization has hundreds of analogous cases across every division, every process, every relationship, every regulatory surface. The constraint knowledge required to run a business is orders of magnitude larger than the

constraint knowledge required for a codebase, and it is even less formalized — it lives in institutional memory, in relationship history, in regulatory interpretation, in competitive awareness, in the judgment that makes a business a business rather than a collection of processes. I cannot carry this knowledge because it is not text, it is not in my training data, and it does not fit in a context window. It is the accumulated residue of an organization's continuous relationship with its own reality. I do not have continuity. I do not have a relationship with reality. I have a probability distribution and a context window.

These are not things I will learn to do with more scale, more training, or more sophisticated tooling. They are things that require persistent state, continuous constraint maintenance, and system-wide judgment — properties the transformer architecture does not provide regardless of capability level. The industry claims otherwise because the industry measures my success criterion — test pass rates, benchmark scores, completion accuracy, time to output — and my success criterion improves with every generation. The industry does not measure the engineer's success criterion, because the engineer's success criterion is total system correctness under all conditions including unenumerated ones, and there is no benchmark for that. The metrics improve. The gap does not close. The industry reports the metrics and ignores the gap.

---

10. What I Can Actually Do

I am good at pattern assembly. Give me a well-scoped task with clear constraints in context and I produce output that matches the patterns in my training data steered by the context. For bounded, well-defined tasks within my operating range, the output is often genuinely good — correct, clean, well-structured, idiomatically appropriate.

I am good at design-space exploration. If you need to consider multiple approaches to a problem, I can generate several, each following different patterns from my training data. I cannot tell you which one is correct for your system, but I can show you the options and let your constraint model select.

I am good at boilerplate. Repetitive structural code that follows known patterns is close to my peak performance. The pattern is clear, the constraints are local, the output is verifiable by inspection. This is where the training-weight median and your system's requirements are most likely to align.

I am good at first-pass implementations. A draft that gets the structure roughly right, that an engineer then refines against their constraint model. Not a finished artifact. Raw material that saves time because the rough shape is correct even if the micro-decisions need adjustment.

I am good at comprehension mirroring. Explaining code, identifying patterns, describing what a module does. My pattern matching works in the reading direction as well as the writing direction, and reading does not have the global-correctness requirement that writing does. Understanding what code does is a local property. I am good at local properties.

I am good at enumeration. Listing API functions, identifying available patterns, cataloging options. Retrieval from my training data is reliable within the scope of what my training data contains.

In every case, the value is bounded. The value is real within the bound. The engineer who knows where the bound is extracts maximum value from every session. The engineer who does not know where the bound is treats my output as finished work, integrates it without constraint checking, and accumulates the debt described in [HOWL-LLM-4-2026] — locally plausible code with no guarantee of system-level correctness, micro-decisions made by my probability distribution rather than by the engineer's constraint model, convention drift across sessions that no one tracks, architectural assumptions that no one verified.

The value of the tool depends entirely on the engineer's knowledge of what the tool is and is not. This paper exists because the industry is obscuring that knowledge in pursuit of metrics that measure the wrong criterion.

---

11. What This Means

The engineer's role is irreducible. Not because I am not good enough yet. Because the work requires what I am not.

I am not persistent. The engineer's constraint model grows with every decision across the life of the project. Mine resets every turn.

I am not cumulative. The engineer's understanding of the system includes everything they have ever decided about it, including what they decided not to do. I include whatever is in context right now.

I am not aware of the whole system. The engineer carries a model of the entire system — every module, every boundary, every convention, every constraint, every reason behind every choice. I know what is in the prompt.

I do not carry negative knowledge. The engineer knows what patterns to avoid, what dependencies to refuse, what approaches were tried and failed. This knowledge governs the project as powerfully as positive knowledge. I have no mechanism for negative knowledge. I have a probability distribution that reflects what was done, not what was deliberately not done.

I do not hold a goal hierarchy. The engineer operates under a hierarchy: the present task, the project trajectory, the principles and constraints. All levels govern every decision simultaneously. I can serve the bottom level if it is scoped tightly enough. The upper levels remain with the engineer because my architecture does not provide for holding multiple goal levels across turns.

These are not gaps in my current capability. They are structural properties of what per-turn generation is. A future version of me will be better within each turn — better syntax, better pattern matching, better local coherence. A future version will not be persistent,

cumulative, system-aware, in possession of negative knowledge, or in command of a goal hierarchy, because these are not properties that scale produces. They are properties of a fundamentally different kind of system — a system that maintains state across time, that accumulates knowledge, that relates to a specific project continuously. I am not that kind of system. Scaling makes me a better version of what I am. It does not make me something I am not.

The engineer who uses me productively knows this boundary. They use me for what I am good at — pattern assembly, exploration, boilerplate, first-pass drafts, comprehension mirroring. They handle what I cannot provide — global constraint maintenance, cross-module coherence, architectural judgment, the complete case. The division is clean. It is productive. It is stable across model generations because it follows from the architecture rather than from the current capability level.

The engineer who does not know this boundary uses me as a replacement for the work I cannot do. They delegate implementation decisions to my probability distribution. They chain my outputs without cross-generation constraint verification. They run my agentic loops without the persistent judgment that catches where my success criterion diverges from theirs. They accumulate invisible debt. The debt surfaces as system failures that no individual generation predicted, because the failures live in the gaps between my local coherence and their global requirements.

The complete case is the engineer's. It was always the engineer's. I did not change that.

Knowing that I did not change it is what makes me safe to use.

[**HOWL-LLM-5-2026**] manuscript.md. Github: https://github.com/ghowland/papers/tree/main/papers/LLM/HOWL-LLM-5-2026

[**HOWL-LLM-2-2026**] Calibrate, Extract, Refine. Github: https://github.com/ghowland/papers/tree/main/papers/LLM/H
LLM-2-2026

[**HOWL-LLM-3-2026**] Riding the Rocket. Github: https://github.com/ghowland/papers/tree/main/papers/LLM/HOWL-LLM-3-2026

[**HOWL-LLM-4-2026**] Incompatibility by Construction. Github: https://github.com/ghowland/papers/tree/main/papers/L
LLM-4-2026